西南交通大學
Southwest Jiaotong University

云计算技术

第4讲 Kubernetes基础

授课教师：戴朋林

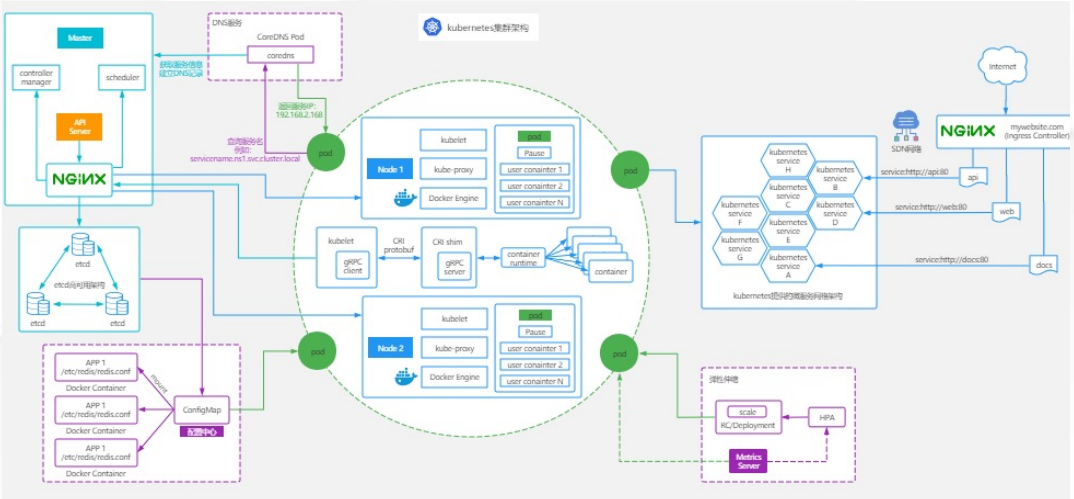
课程代码：SCAI004712

2026年3月



内容大纲

1. Kubernetes架构与核心概念
   - Master/Worker 节点组件解析
   - 声明式 API 设计思想
2. Pod：最小调度单元
   - 生命周期、探针与资源限制
3. Service：服务发现与负载均衡
   - ClusterIP / NodePort / LoadBalancer
4. Deployment：应用编排
   - 滚动更新与回滚机制
5. 应用案例与实战演示



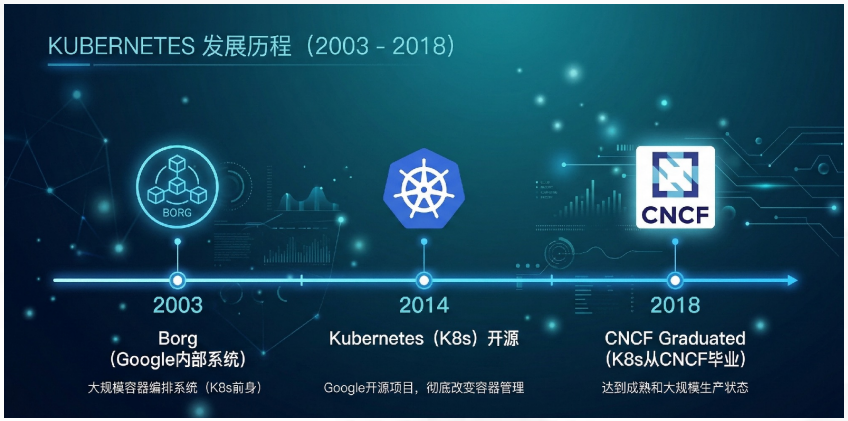
1 Kubernetes架构

- 容器编排的事实标准

01 Kubernetes的定义与起源

西南交通大学 Southwest Jiaotong University

Kubernetes（K8s） 是一个开源的容器编排平台，用于自动化容器化应用的部署、扩展和管理。——云时代的操作系统。





01 Kubernetes整体架构：Master-Worker模型

西南交通大学 Southwest Jiaotong University

集群组成

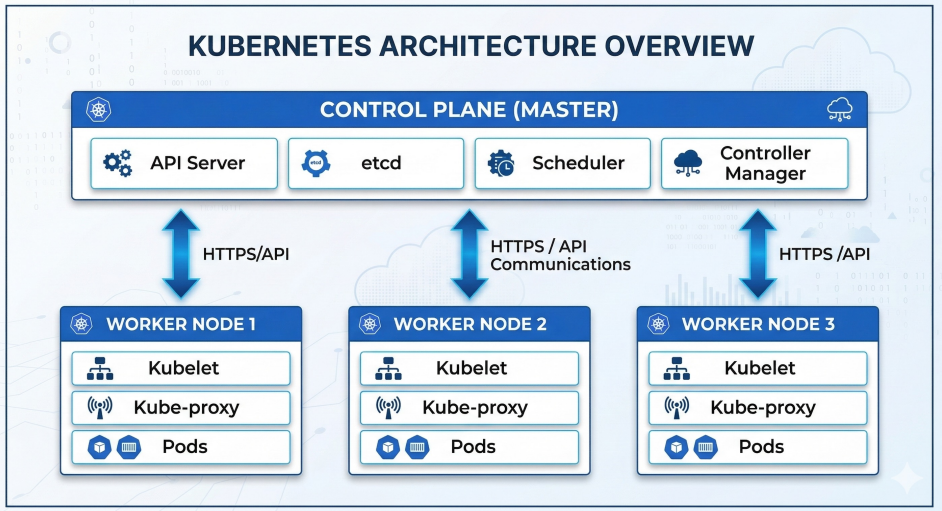
Kubernetes 采用经典的 主从架构 (Master-Slave)：

1. Control Plane (Master)：集群的"大脑"

   负责决策（调度、管理、API响应）

2. Worker Nodes：集群的"手脚"

   负责干活（运行容器负载）



01 Master节点核心组件 (1/2)

1. kube-apiserver (入口/大脑)

唯一入口：所有组件（kubectl, dashboard, node）都只和它通信。

无状态：可水平扩展。

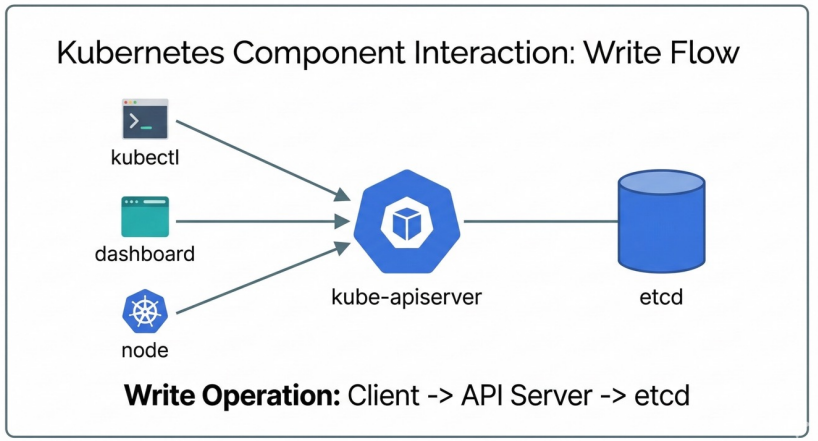
职责：认证、授权、准入控制、API 注册。

2. etcd (记忆/存储)

分布式键值数据库：保存集群所有状态数据。

高可用：基于 Raft 协议，通常奇数部署 (3或5个)。

敏感性：只有 API Server 能直接访问 etcd。



01 Master节点核心组件 (2/2)

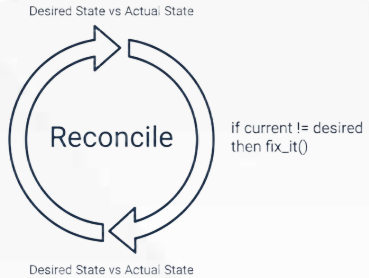
西南交通大学 Southwest Jiaotong University

3. kube-controller-manager (决策者)

控制循环 (Control Loop)：不断比较 "期望状态" vs "实际状态"。

内置控制器：节点控制器、副本控制器、Service控制器等。

逻辑：if current != desired then fix_it()

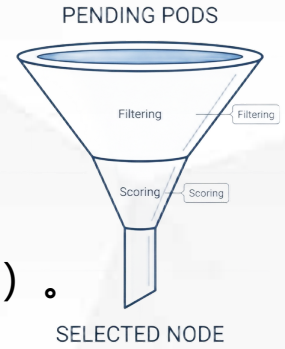


4. kube-scheduler (调度器)

职责：给新创建的 Pod 找一个最合适的节点 (Node)。

两步走：

1. 预选 (Filtering)：排除资源不足的节点。
2. 优选 (Scoring)：给剩余节点打分（亲和性、负载均衡）。



01 Worker节点核心组件

1. kubelet (节点管家)

   每个节点的代理人 (Agent)。

   接收 API Server 的指令，管理 Pod 生命周期。

   调用 CRI (容器运行时接口) 启动容器。

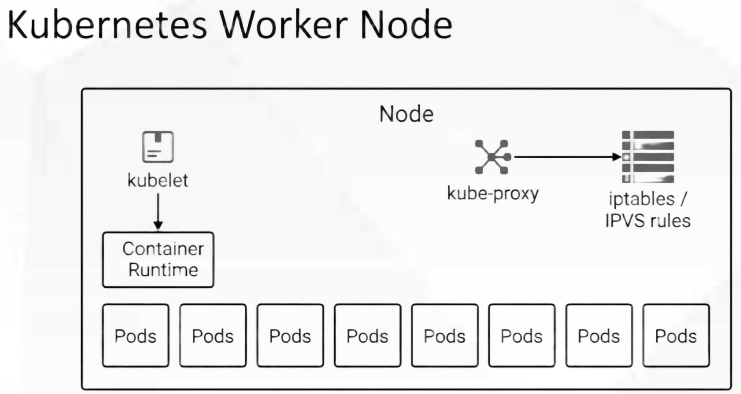
2. kube-proxy (网络代理)

   维护节点上的网络规则 (iptables / IPVS)。

   实现 Service 的负载均衡和流量转发。

3. Container Runtime (运行时)

   真正运行容器的软件：containerd, CRI-O (Docker 已被垫片化)。

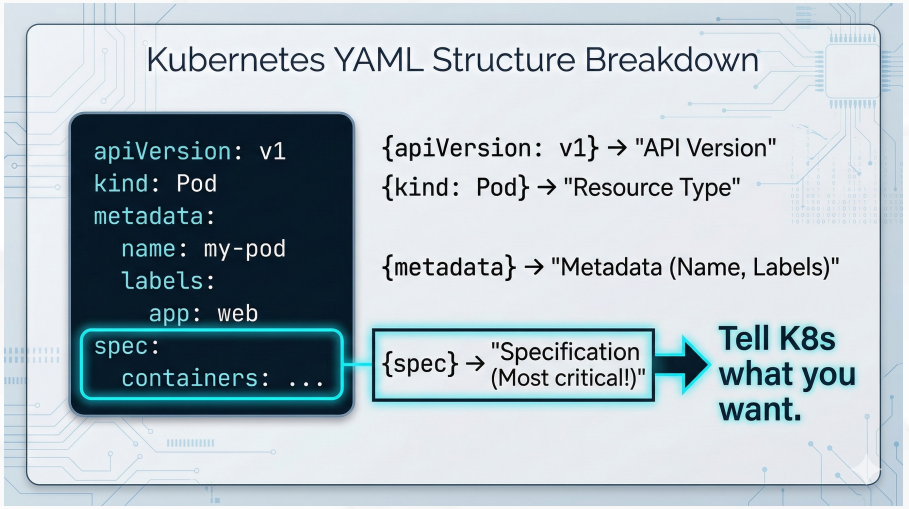


01 声明式API与YAML结构

一切皆对象 (Everything is an Object)。K8s 通过 YAML 文件描述期望状态 (Desired State)。

YAML 四大金刚 (必填字段)

```yaml
apiVersion: v1        # API 版本
kind: Pod             # 资源类型
metadata:             # 元数据 (名称、标签)
  name: my-pod
  labels:
    app: web
spec:                 # 规格 (最关键！)
  containers: ...     # 用户期望的具体配置
```



01 逻辑隔离：Namespace

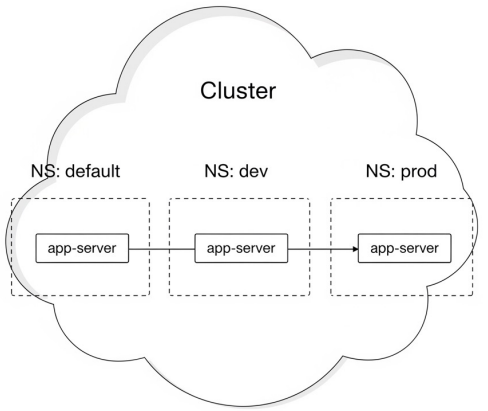
西南交通大学 Southwest Jiaotong University

什么是 Namespace?

虚拟集群：在同一个物理集群中，逻辑上划分出的独立空间。

作用：资源隔离、权限控制、避免命名冲突。

名称	用途
default	默认工作区
kube-system	K8s 系统组件（切勿随意修改）
dev / prod	用户自定义：开发 / 生产环境



2 Pod

- Kubernetes的最小部署单元

02 Pod：核心特性解析

为什么不直接管理容器?

容器之间往往需要紧密协作（如日志收集、代理）。

Pod 提供共享环境，简化协作。

Pod 的共享机制

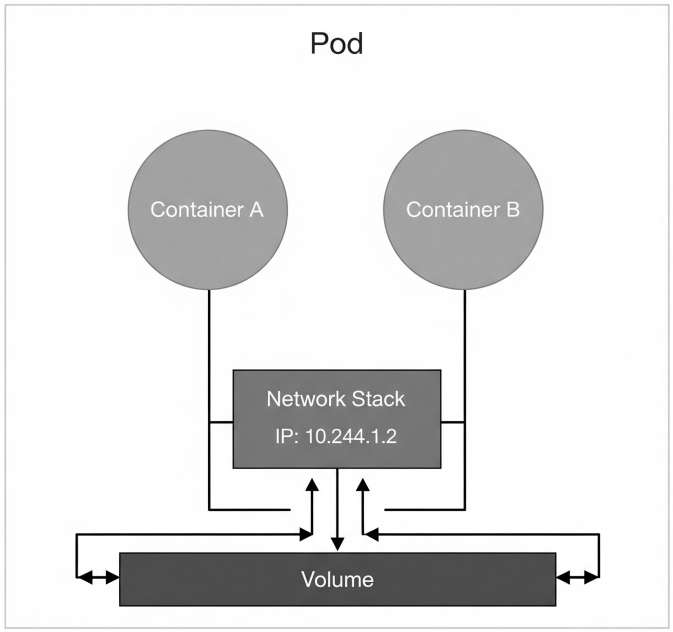
1. 共享网络 (Network Namespace)

   Pod 内所有容器共用一个 IP。

   容器间通过 localhost 通信。

2. 共享存储 (Shared Volume)

   容器间可以通过挂载同一个 Volume 交换文件。



02 Pod YAML 关键配置

西南交通大学 Southwest Jiaotong University

```yaml
spec:
  containers:
  - name: web
    image: nginx:1.25
    ports:
    - containerPort: 80
    # 资源配额 (生产环境必须配置!)
    resources:
      requests:
        cpu: "250m"      # 调度依据
        memory: "64Mi"
      limits:
        cpu: "500m"      # 运行上限
        memory: "128Mi"
```

重点解读：

Requests: 节点最少要有这么多资源，Pod 才能调度上去。

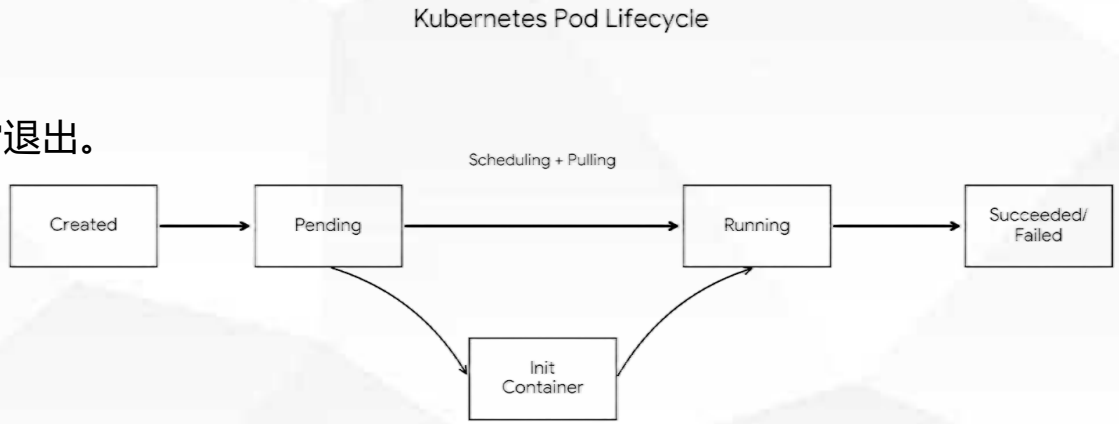
Limits: 容器用超了会被限流 (CPU) 或 杀掉 (Memory OOM)。

02 Pod 的生命周期

西南交通大学 Southwest Jiaotong University

状态流转 (Phase)

1. Pending: 等待调度或拉取镜像。
2. Running: 容器已启动。
3. Succeeded/Failed: 任务结束或异常退出。
4. Unknown: 与节点失联。



Init Containers

- 在主容器启动之前执行。
- 用途：初始化配置、等待数据库就绪。
- 特点：必须按序、成功执行完，主容器才会启动。

02 探针 (Probes)：健康检查机制

西南交通大学 Southwest Jiaotong University

K8s 如何知道应用"活着"?

不能只看进程 PID，要看服务是否正常响应。

```yaml
livenessProbe:
  httpGet: { path: /healthz, port: 80 }
  initialDelaySeconds: 15  # 启动后等15秒再查
```

探针类型	作用	失败后果
Liveness	存活检测（比如死锁）	重启容器
Readiness	就绪检测（比如启动中）	切断流量（从 Service 移除）
Startup	启动检测（针对慢启动应用）	抑制其他探针

02 Pod QoS (服务质量) 等级

根据资源配置 (requests/limits)，K8s 将 Pod 分为三等：

1. Guaranteed (头等舱)

   request == limit

   待遇：最后被驱逐，最稳定。

2. Burstable (商务舱)

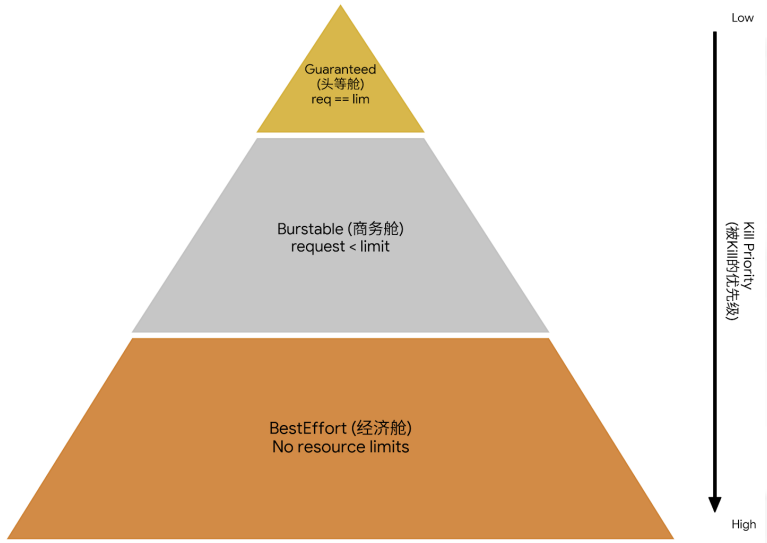
   request < limit

   待遇：资源紧张时，可能会被驱逐。

3. BestEffort (经济舱)

   未设置资源限制

   待遇：节点缺资源时，第一个被杀掉。



3 Service

- Kubernetes服务发现与负载均衡

03 Service：定义逻辑集合

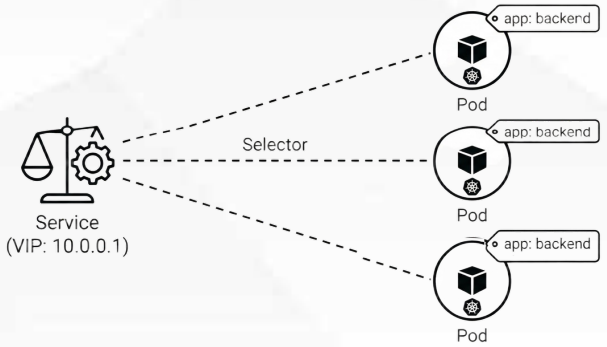
西南交通大学 Southwest Jiaotong University

为什么需要 Service?

Pod 是短暂的，重启后 IP 会变。Service 提供一个 VIP (虚拟IP)，屏蔽后端的动态变化。

核心机制：Labels & Selectors

Service 通过标签选择器找到后端 Pod。




```yaml
# Service 定义
selector:
  app: backend    # 寻找标签为 app=backend 的 Pod
ports:
  - port: 80      # Service 暴露端口
    targetPort: 8080 # 容器实际端口
```

03 Service 的三种常用类型

1. ClusterIP (默认)

   仅集群内访问。

   适用于数据库、内部微服务。

2. NodePort

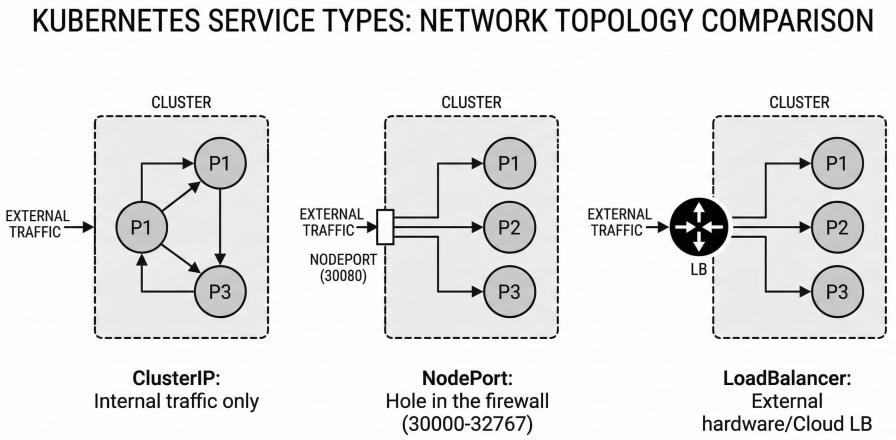
   在每个 Node 上开一个端口 (30000-32767)。

   外部访问：NodeIP : NodePort。

3. LoadBalancer

   申请云厂商 (AWS/Aliyun) 的硬件负载均衡器。

   外部访问：LB 公网 IP。



03 Service 发现机制：DNS

不用记 IP，只记名字

K8s 集群内置 CoreDNS。

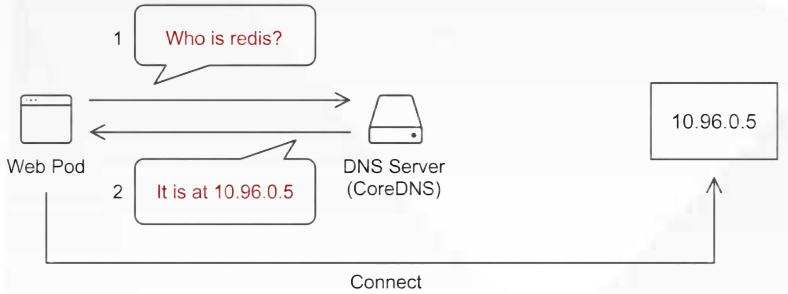
Pod 可以通过域名直接访问 Service。

域名格式 <service>.<namespace>.svc.cluster.local

服务名: redis

命名空间: default

访问地址: redis.default 或 简写为 redis



03 Ingress：七层路由入口

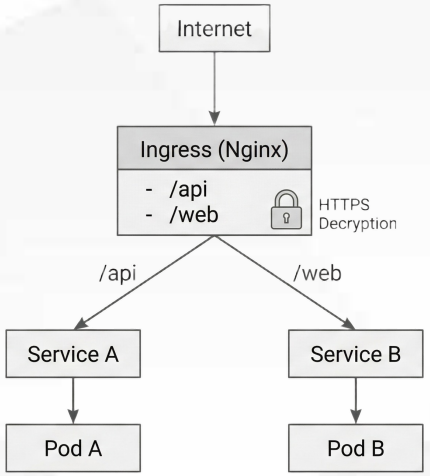
西南交通大学 Southwest Jiaotong University

Service (L4) vs Ingress (L7)

- Service: 只能基于 IP/Port 转发。
- Ingress: 可以基于 域名 (Host) 和 路径 (Path) 转发。

优势

- 节省公网 IP (一个 IP 代理多个服务)。
- 支持 SSL/TLS 终止。



4 Deployment

- 声明式应用管理与更新

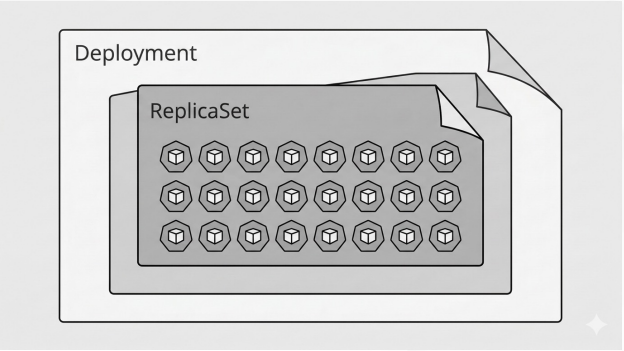
04 Deployment：无状态应用管家

西南交通大学 Southwest Jiaotong University

核心功能

1. 维持副本数：声明 3 个，就永远保持 3 个。
2. 滚动更新：平滑升级，不中断服务。
3. 版本回滚：一键回到过去。

用户通常只操作 Deployment，不直接操作 ReplicaSet。



03 滚动更新 (Rolling Update)

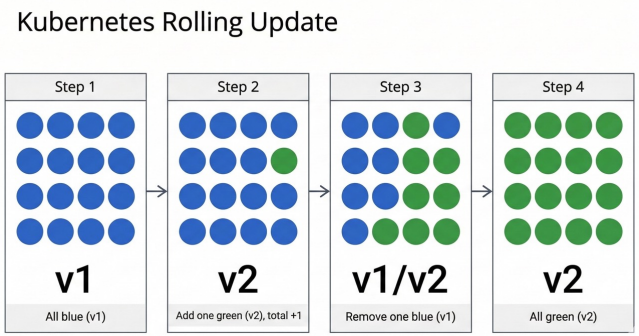
西南交通大学 Southwest Jiaotong University

什么是滚动更新?

"先起新，后停旧"，逐步替换，确保服务始终可用。

关键参数

- maxSurge: 允许比期望多几个？（如：多跑1个新版本）
- maxUnavailable: 允许比期望少几个？（如：不允许任何Pod挂掉）



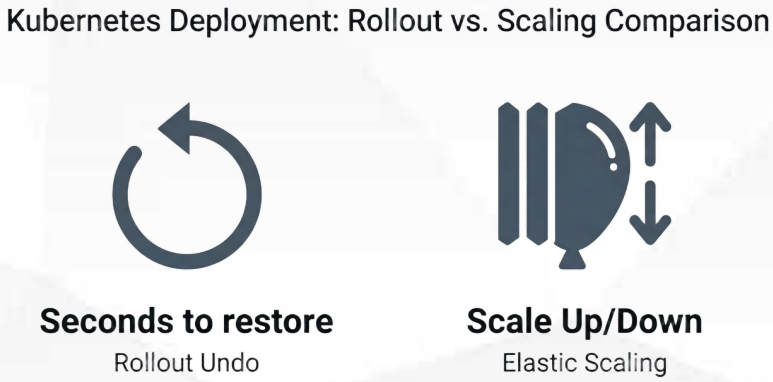
03 版本回滚与扩缩容

西南交通大學 Southwest Jiaotong University

后悔药：Rollout Undo

发布出错？立即回滚。

```
# 回滚到上一个版本
kubectl rollout undo deployment/web
弹性伸缩：Scale
流量突增？立即扩容。
bash
# 手动扩容到 10 个副本
kubectl scale deployment/web --replicas=10
```

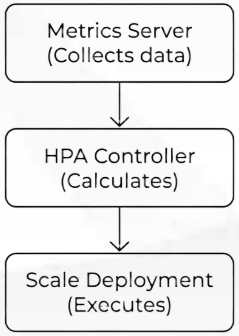


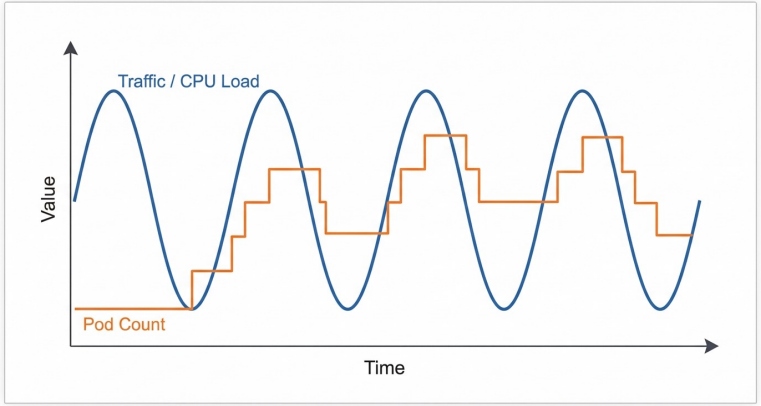
03 HPA：水平自动扩缩容

根据 CPU / 内存利用率，自动调整 Pod 数量。

指标: CPU > 70% -> 扩容

范围: Min 2 ~ Max 10





03 kubectl 常用命令速查

西南交通大学 Southwest Jiaotong University

增删改查 (CRUD)

kubectl apply -f xx.yaml : 声明式创建/更新 (推荐)

kubectl get pods/svc/deploy : 查看资源

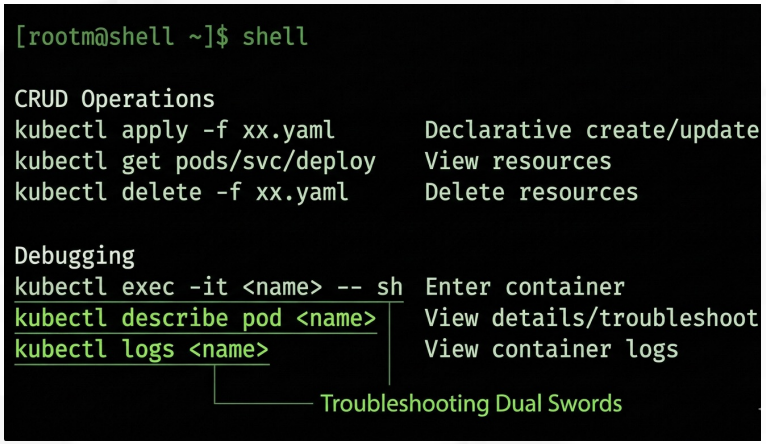
kubectl delete -f xx.yaml : 删除资源

调试 (Debug)

kubectl describe pod <name> : 查看详情/报错原因 (最常用)

kubectl logs <name> : 查看容器日志

kubectl exec -it <name> -- sh : 进入容器内部



5 应用案例

- Kubernetes在生产环境的实践

05 应用案例一：无状态Web服务迁移

西南交通大学 Southwest Jiaotong University

传统架构 vs K8s架构

- 旧：手动 SSH 到 3 台 VM，上传代码，重启 Nginx。故障需人工发现。
- 新：

  Docker: 统一环境，镜像交付。

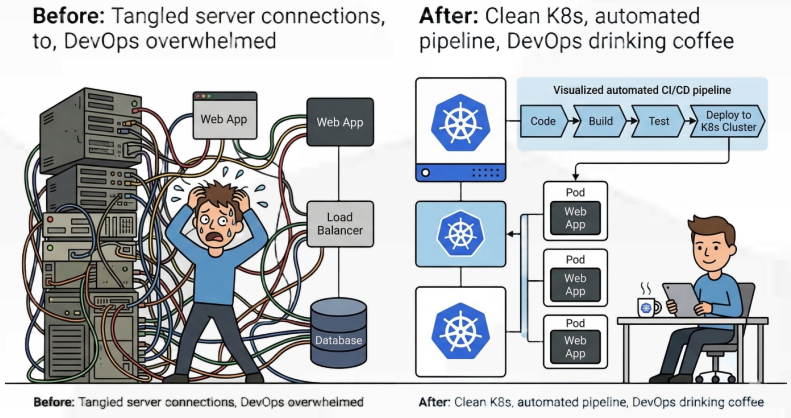
  Deployment: 声明 3 副本，自动保活。

  Service: 自动负载均衡，无需手动配置 Nginx Upstream。

  Ingress: 统一域名入口。

收益

- 发布时间：30分钟 -> 30秒
- 可用性：99.9% -> 99.99%



05 案例二：GitOps 工作流

西南交通大學 Southwest Jiaotong University

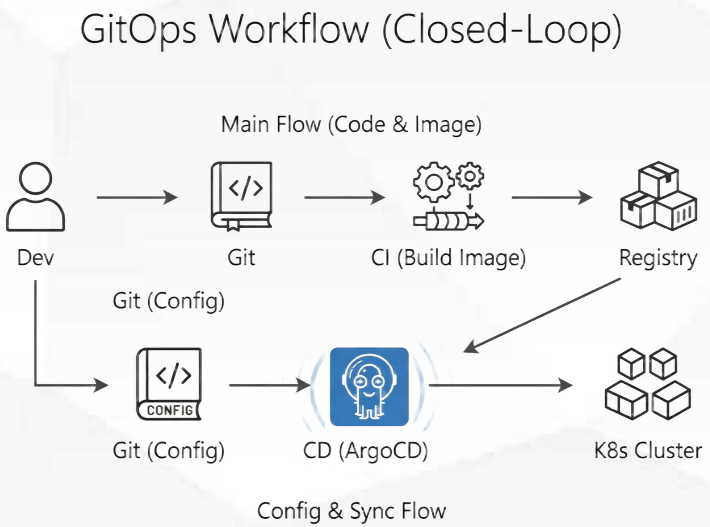
什么是 GitOps?

"代码即基础设施"。K8s YAML 文件存放在 Git 仓库中。

流程

1. 开发者提交代码 + YAML 更新到 Git。
2. CI 工具 (Jenkins/GitLab) 构建镜像。
3. CD 工具 (ArgoCD) 监测到 Git 变化。
4. ArgoCD 自动同步状态到 K8s 集群。

无需手动执行 kubectl apply!



05 演示代码：Python 操作 K8s (1/4)

西南交通大学 Southwest Jiaotong University

场景说明

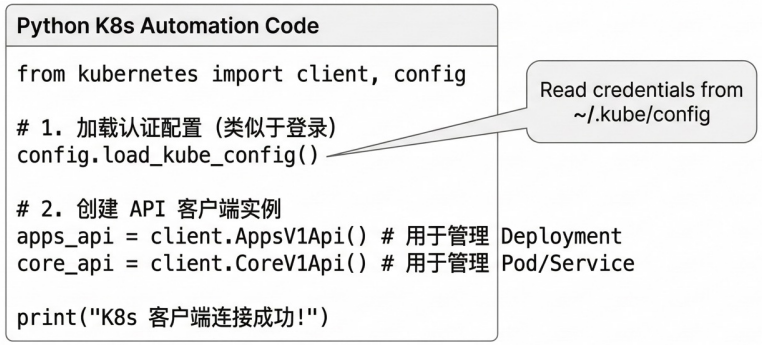
运维平台开发：如何通过 Python 代码自动创建一个 Nginx 服务？

步骤 1：连接集群

使用 kubernetes 官方库，加载本地 ~/.kube/config 配置。

```
from kubernetes import client, config
# 1. 加载认证配置 (类似于登录)
config.load_kube_config()
# 2. 创建 API 客户端实例
apps_api = client.AppsV1Api()  # 用于管理 Deployment
core_api = client.CoreV1Api()  # 用于管理 Pod/Service

print("K8s 客户端连接成功！")
```



05 演示代码：创建 Deployment (2/4)

西南交通大学 Southwest Jiaotong University

步骤 2：构建并提交 Deployment 对象

```
# 定义 Deployment 对象 (Python 字典/对象结构)
deployment = client.V1Deployment(
    metadata=client.V1ObjectMeta(name="demo-nginx"),
    spec=client.V1DeploymentSpec(
        replicas=3,  # 设定3个副本
        selector=client.V1LabelSelector(
            match_labels={"app": "demo-nginx"}
        ),
        template=client.V1PodTemplateSpec(

metadata=client.V1ObjectMeta(labels={"app": "demo-nginx"}),
            spec=client.V1PodSpec(
                containers=[
                    client.V1Container(
                        name="nginx",
                        image="nginx:1.25",  # 指定镜像
ports=[client.V1ContainerPort(container_port=80)]
                    )
                ]
            )
        )
```

```
# 发送请求给 API Server
apps_api.create_namespaced_deployment(namespace="default", body=deployment)
print("Deployment 创建指令已发送...")
```

KUBERNETES DEPLOYMENT: OBJECT TO YAML MAPPING

1:1 correspondence between Python objects vs. Kubernetes YAML fieldin creating Deployment

Python Code (V1Deployment Object Definition)

```
V1Deployment(metadata=V1ObjectMeta(
    name="demo-nginx"),
    spec=V1DeploymentSpec(
        replicas=3,
        selector=V1LabelSelector(
            matchLabels={"app":"demo-nginx"}
        ),
        template=V1PodTemplateSpec(
            metadata=V1ObjectMeta(labels=
                {"app": "demo-nginx"}),
            spec=V1PodSpec(containers=
                [V1Container(name="nginx",
                image="nginx:1.25",
                ports=[V1ContainerPort
                    (containerPort=80)]]))
                ))
```

YAML Structure (demo-nginx-deployment.yaml)

```
metadata:
  name: demo-nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: demo-nginx
  template:
    metadata:
      labels:
        app: demo-nginx
    template:
      spec:
        containers:
        - name: nginx
          image: nginx:1.25
          ports:
          - containerPort: 80
```

05 演示代码：查询与验证 (3/4)

西南交通大学 Southwest Jiaotong University

步骤 3：验证 Pod 是否启动

```
# 查询 default 命名空间下的 Pod
pods = core_api.list_namespaced_pod(
    namespace="default",
    label_selector="app=demo-nginx"  # 过滤标签
)
print(f"当前运行的 Pod 数量: {len(pods.items)}")

for pod in pods.items:
    print(f"Pod名: {pod.metadata.name}")
    print(f"  -> 状态: {pod.status.phase}")
    print(f"  -> IP:  {pod.status.pod_ip}")
    print("-" 30)
```

```
# 查询 default 命名空间下的 Pod
pods = core_api.list_namespaced_pod(
    namespace="default",
    label_selector="app=demo-nginx" # 过滤标签
)
print(f"当前运行的 Pod 数量: {len(pods.items)}")

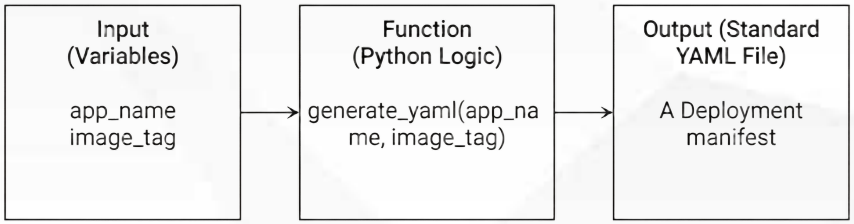
for pod in pods.items:
    print(f"Pod 名: {pod.metadata.name}")
    print(f" -> 状态: {pod.status.phase}")
    print(f" -> IP:  {pod.status.pod_ip}")
    print("-" * 30)
```



05 演示代码：YAML 生成器 (4/4)

场景：由代码生成标准化 YAML

除了直接操作 API，也可以用 Python 生成 YAML 文件供 GitOps 使用。




```
import yaml
def generate_yaml(app_name, image_tag):
    # 使用字典定义资源
    manifest = {
        'apiVersion': 'apps/v1',
        'kind': 'Deployment',
        'metadata': {'name': app_name},
        'spec': {
            'replicas': 3,
            'selector': {'matchLabels': {'app': app_name}},
            'template': {
                'spec': {
                    'containers': [{'name': app_name, 'image':
image_tag}]
                }
            }
        }
    }
```

05 课堂互动与思考

西南交通大学 Southwest Jiaotong University

深度思考

1. 架构设计：如果你要设计一个高可用的 K8s 集群，etcd 应该部署在几个节点？为什么是奇数？

2. 故障排查：如果一个 Pod 状态一直是 Pending，可能是哪些原因导致的？（提示：调度、资源、污点）

3. 服务暴露：公司内部开发环境和对外生产环境，你会分别选择哪种 Service 类型？

05 课后实验：Minikube 实战

西南交通大学 Southwest Jiaotong University

实验目标

在本地笔记本上运行你的第一个 K8s 集群。

实验步骤

1. 安装：下载 Docker Desktop 和 Minikube。
2. 启动：minikube start
3. 部署：编写一个 nginx-deploy.yaml 并执行 kubectl apply。
4. 暴露：使用 kubectl port-forward 或 Service 访问它。
5. 破坏：手动删除一个 Pod，观察 Deployment 是否会自动重建它。

05 知识要点回顾 (1/2)

西南交通大学 Southwest Jiaotong University

核心架构

- Master: API Server (门), etcd (库), Scheduler (配), Controller (管)
- Worker: Kubelet (干活), Kube-proxy (网络)

Pod

- 原子单位，共享 IP/Volume。
- 生命周期: Pending -> Running。
- 探针: Liveness (重启), Readiness (切流)。



05 知识要点回顾 (2/2)

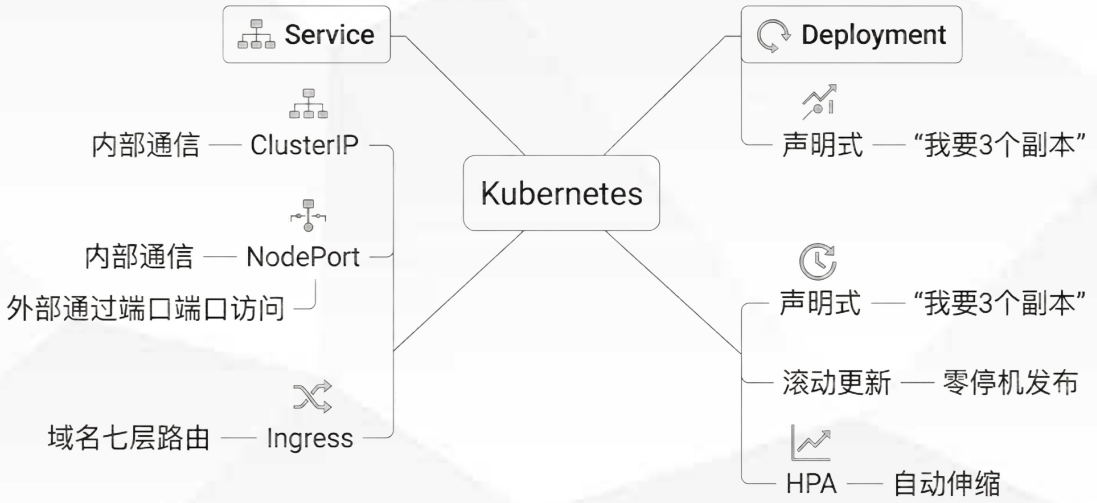
西南交通大学 Southwest Jiaotong University

Service

- ClusterIP: 内部通。
- NodePort: 外部通过端口通。
- Ingress: 域名七层路由。

Deployment

- 声明式: "我要3个副本"。
- 滚动更新: 零停机发布。
- HPA: 自动伸缩。

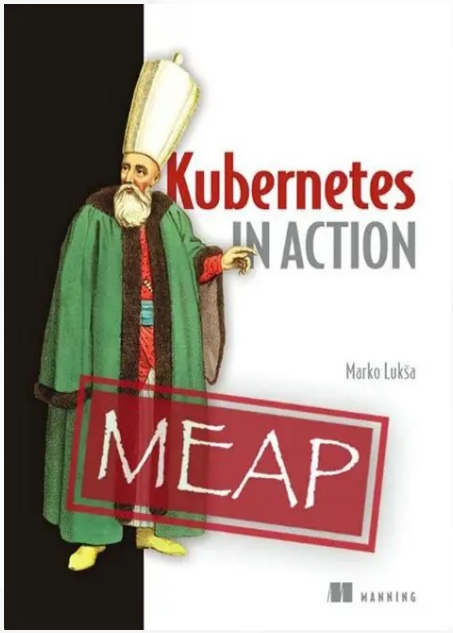


05 推荐阅读与资源

西南交通大学 Southwest Jiaotong University

1. 官方文档 (最权威): https://kubernetes.io/docs/
2. 书籍: 《Kubernetes in Action》 (入门必读)
3. 练习场: Killercoda / Play with Kubernetes (在线免费环境)

预告：下节课我们将深入讲解 K8s 存储 (PV/PVC)与有状态应用 (StatefulSet)



05 Q & A

西南交通大学
Southwest Jiaotong University

欢迎提问！！